

Virtualization

Virtualization

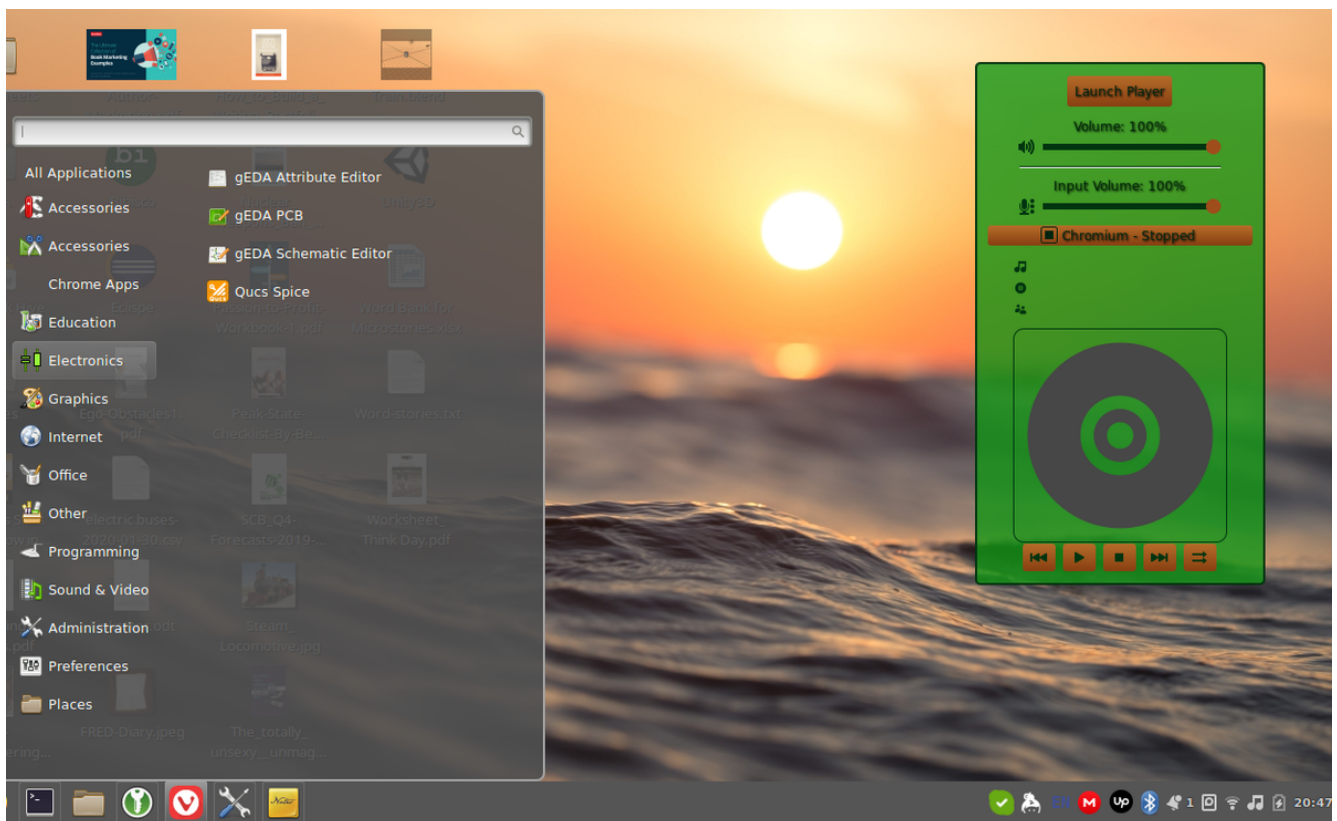
© 2017-2024, Pum Walters, HvA/MITW

1. Introduction

This document offers an introduction to virtualization and Docker. The document is based on lecture notes from 2017, and it may not be up to date in all places.

1.1. Operating Systems

For the average computer user, the 'operating system' is the visual appearance of their computer. How windows and buttons look, where the dock or taskbar are, etcetera. Strictly speaking, that is the windowing system, which is part of the OS. The average user doesn't perceive the difference because in the most common OS's the windowing system is embedded. Mac OSX and Microsoft Windows have their own built-in fine-tuned windowing system. In Linux this relation isn't fixed. At a lower level Linux often uses the X11 windowing system, which offers the basics for GUI elements and behavior. On top of that many different desktop environments can be plugged in, such as KDE, Gnome or IceWM, resulting in significantly different user experiences.



For us experts, the central role of the operating system is to manage hardware and software resources as efficiently as possible. **The OS 'owns' all resources and lends them out to processes that need them.** The OS covers four main areas

- * One or more CPU's, possibly each having multiple cores. CPU's aren't handed to processes in the hope they will quickly be returned. Instead, processes only get the CPU for a limited slice of time.

The task of determining which process gets the CPU how long is called **scheduling**.

- * **Memory.** Memory is usually done on a first-come first-serve basis until all available memory is exhausted. Then, recently unused memory is written to disk, allowing the physical memory to be reused. Whenever a previous content is needed again, other data is written to disk, and the memory it was in is refreshed from disk to reflect that previous content.
- * **Disks.** In several layers, the capabilities and limitations of huge raw data storage is turned into flexible internal information system. Note that memory and disks are part of a bigger resource: **storage**.
- * The final task for the OS is the management of all IO devices and sub-systems, ranging from keyboard and mouse to Graphical cards, which are computers in their own right.

1.2. Virtualization

From this description it is clear that one computer cannot simply run two operating systems at once. It would be like two captains on a ship, each thinking they rule everything.



It isn't realistically imaginable to consider adapting two OS's so that they can run concurrently on shared hardware. Each OS is geared towards managing all resources full time and as efficiently as possible. Taking another unpredictable OS into account would be folly.

And why would we?

The answer to that question is simple: it has so many many advantages (we will go into the advantages in a moment).

Because it is so useful, the issues have been resolved: **virtualization allows multiple operating systems to use the same hardware**.

The crux is an asymmetric setup. One OS 'owns' all hardware (hardware and OS are jointly called **host**). The host offers (the use of) resources to another OS (called the **guest**), in such a way that the other OS may not even be aware that it doesn't own all hardware all the time; it is only aware of what it has. Easy? Not at all.

1.2.1. Emulation

The naive simplest form of Virtualization is Emulation. The host has an interpreter, which interprets all guest OS activity one step at a time. This is similar to a micro-code interpreter in a CPU or a language interpreter you may have heard of.

This works, but there is one glaring issue: the layer of interpretation slows the guest down so much that it can't practically be used.

Over the years, many system engineers have improved upon this bleak outset.

One key improvement has been made possible because most modern processors support the same architecture: X86 (-64). Because of this, the host and guest can run on the same processor, and with a bit of (now broadly available) hardware support, the guest can run directly on the physical CPU.

Today, a virtualized guest runs 5-10% slower overall than the host would directly, and uses 10-20% more memory than the host would directly. So for a small price, all advantages are there.

1.3. Advantages of Virtualization

Here are some advantages to virtualization:

1.3.1. Utilization of resources; load averaging

A physical server that doesn't have much to do is occupying hardware needlessly. Virtualizing the server means that other guests can use the same (host) hardware.

Servers that are only needed some of the time are another example. Backup servers are often used during the night, while office servers are used during the day. They can share the same hardware, lowering the costs.

Using containers (which we will discuss later), a normal server can easily run tens if not hundreds of guests, as long as they are idle a large part of the time.

1.3.2. Maintenance

If a physical server breaks down or otherwise needs hardware support, it is unavailable for the end users. But a virtualization guest can simply be moved to another host. Planned support while the guest remains running is easy (moving a guest is a matter of seconds, if that). Unplanned support (host crashes) is more complicated, but solutions for hot-swappable guests have been developed.

A second aspect is software support. Large organisation can maintain a guest template which is rolled out for all users. The template doesn't contain user files, but does contain up-to-date applications etcetera. Different ways to do this will be discussed.

1.3.3. Security

Assuming the host is secure, guests are separated as if they run on separate hardware. This way, separate servers (say, for separate customers or departments) can use the same hardware with strict guarantees about security.

1.3.4. Other Considerations

Replace many small servers by a few powerful servers:

- * lower overall hardware costs
- * lower energy footprint
- * lower risk profile (disaster recovery, downtime)

- * separate hardware management from software management (VM's can be moved easily)
- * agility, flexibility, scalability

Also use in software development, education etcetera to use different platforms without the cost.

One concern: software licenses.

2. Virtualization

2.1. Area of Attack

Virtualization can occur in different areas, using different engineering techniques.

- * Server-level -- hypervisors.

A **hypervisor** is an emulation layer aimed at supporting virtualized guests.

- * Type 1

The hypervisor is a full OS running directly on the hardware (usually without windowing system and other end-user oriented functionality). For example, in Windows editions, the Hyper-V hypervisor is included as a service, which, when turned on, allows the host OS to act as a hypervisor for similar Windows guests.

- * Type 2

In this approach the hypervisor runs as an application on top of the host OS. More hardware is emulated so guests do suffer a performance loss. In practice the CPU isn't emulated and runs at full speed, but system calls accrue a penalty.

- * OS-level -- Containers

In a container, the host and guest OS's are identical, and the guest uses the hosts code, libraries and system calls. Only configurations and user data need to be separated.

- * Paravirtualization

In paravirtualization the guest OS is adapted to be aware of the fact that it is virtualized. This leads to a significant reduction in the virtualization penalty.

- * Desktop

Desktop virtualization separates the desktop and available applications from the physical device we are using. Any thin client without disk storage can be used to access a user's files and setup.

- * Application

Software is virtualized, appearing on a user's device as an actual application, but stored elsewhere on a server, simplifying software support and reducing disk and maintenance costs.

2.1.1. Type 1, Type 2

- * A Type 1 or bare-metal hypervisor is an OS which directly acts as a hypervisor
 - * KVM, Red Hat RHEV, XenServer, Hyper-V, ESXi (vSphere)
- * A type 2 hypervisor is a hypervisor which runs as a user program on top of some OS.
 - * VirtualBox, VMWare (workstation, player, fusion), Parallels, ...

2.1.2. Bare metal

If the host and guest are similar, the integration gets better.

For instance, in Windows the Hyper-V service can be turned on or off as if it is a type-2 hypervisor, but in fact it is a type-1 hypervisor which then runs the native Windows OS as a VM.

For instance, RancherOS is a linux OS which runs only the kernel as Type-1 hypervisor (occupying 80 mb) and runs everything else in VMs on top of that.

2.2. (Some) Kinds of Virtualization

- * Application-Server Virtualization
- * Desktop Virtualization
- * Presentation Virtualization
- * User Virtualization
- * Application-Server Virtualization
- * Network Virtualization
- * Storage Virtualization
- * Administrative Virtualization
- * Data Virtualization
- * OS Virtualization <= **mainly interested in this one**
- * Hardware Virtualization
- * Virtualization of dispute resolution

2.2.1. Application Virtualization

- * Application Virtualization (/Portable Applications)
 - * application are stored on a server but use client's ram and cpu to run
 - * advantage: central upgrades etcetera
- * Desktop Virtualization
 - * the entire desktop runs on a server, the client only does keyboard, mouse and display
 - * advantage: central management and dumb clients
- * Presentation Virtualization
 - * likewise: the client only does presentation
- * User Virtualization
 - * Using a centralized user profile

2.2.2. Infrastructure Virtualization

- * (Application-) Server Virtualization
 - * 'Advanced load balancing' distributing applications across servers e.g. high-volume web server
- * Network Virtualization
 - * Creating logical networks (or components) with a structure independent of the underlying
- * Storage Virtualization
 - * Creating logical storage (e.g. RAID, SAN)
- * Administrative Virtualization
 - * Role Based Access Control: users have roles, roles have minimal sufficient rights. The entire admin function is software.
- * Data Virtualization / Database Virtualization

2.2.3. Other Virtualization

- * Virtualization of dispute resolution.
 - * 'Establishing trust by recycling reputation' (e.g. stack exchange)
- * Software virtualization
 - * Emulator
 - * Interpreter

2.3. Virtual Machines (Emulation)

A virtual machine is a set of data which represents the **entire** state of some actual machine: its memory, its registers, all its buffers, its disks and possibly other media, its video memory. Everything.

A program manipulates the VM: use the value stored for the PC to find the instruction located there in the VM. That instruction may use registers, the value of which is stored in the VM. The effect of executing the instruction is computed by an interpreter. The updated PC and the results of the execution (register and/or memory) are updated in the VM.

The entire VM is changed from the state *before* executing the instruction to the state *after* executing the instruction.

Note that in an operating system, a process image can be suspended to disk and swapped in later to be continued as if nothing happened. The idea of storing an entire state isn't new!

2.3.1. Hypervisor

The program that manipulates a VM in this way is called a **Hypervisor**. The hypervisor runs on some system which is called the **host**. Note that the VM may not be 'aware' it is running on simulated hardware. It executes as if on real (albeit slow) hardware.

The VM is called the **guest**

2.4. Virtual Machines (In Practice)

The naive approach described above isn't practical: interpreting instruction in this way slows the guest down to a crawl and renders it almost useless.

Many improvements on this idea exist however.

2.4.1. Direct Execution

The first radical improvement to the naive approach is this:

If the guest and the host have the same CPU, why not execute guest instructions directly in the host CPU. Allowing the guest to run is like moving a process from the ready to the running state.

But wait. Direct execution fails as soon as the guest tries to access its hardware (which isn't there). Fortunately accessing hardware is only allowed in Ring 0 (Protected Mode).

Early virtualization simply captured and emulated traps, where each ring 0 instruction resulted in a trap in which the hypervisor emulated the guest OS call.. But this is still too slow for I/O.

2.4.2. Paravirtualization

Paravirtualization: the guest OS is adapted (eg. recompiled) to use the fact that it is virtualized. This makes sense if the host and guest are similar or agree on an interface.

- * Xen offered paravirtualized device driver to install in a Windows guest on a Xen server
- * VMware also created the VMI
- * Linux includes pv-ops to this end

Paravirtualization is mostly superseded by virtualization extensions.

A mild form are so-called guest-tools: a few drivers for keyboard, mouse and windowing I/O to aid UI.

2.4.3. Intel VT/AMD-V, Intel VTd/AMD IOMMU

Around 2005/2006 extensions were built in processors to allow them to handle guest traps in hardware

Intel VT-x/AMD-V adds new instructions/registers that allow the guest to run in (its) ring 0 but still protect the host.

Intel VTd/AMD IOMMU allows guests to directly use peripheral devices (Ethernet, graphics and disk controllers) through DMA and interrupt remapping

At this point in time a virtual server (unmodified guest with a similar CPU as the host but with an arbitrary OS) runs at practically the same speed as an actual server.

User interaction with an unmodified guest remains cumbersome.

3. Containerization

3.1. CPU

With hardware support, current virtual machines run at near-actual speed. That is, software in the a virtual machine runs at almost the same speed as that software would run on the underlying hardware. The techniques to do so include:

- * A memory-management and paging setup in which one memory manager services both guest and host
- * Hardware support (instructions and registers) to allow the guest to execute ring 0 instructions without the possibility to compromise the host
- * Hardware support to share devices such as USB, mouse and keyboard or to attach them directly to the guest

Virtualization solved the issue of speed, or to put it differently, the effective use of the first resource: CPU.

Now we will look at the second resource: memory.

3.2. Memory

Paravirtualization attempted to improve the speed of virtualization by altering the guest OS, but think of the implications: every change in the guest OS, libraries or user applications implies that the paravirtualized guest must be reverted, updated and paravirtualized again. In a time where daily security patches are common (and crucial), paravirtualization simply isn't appropriate.

So it is important to ensure that a VM is an unmodified version of a real machine, which can keep itself updated where appropriate. But then every VM has its own copy of an OS, applications and all needed data.

A **small** guest typically uses 5-20 GB of disk space, and 0.5-2 GB of Ram.

That means that a decent server can host around ten VMs.

If they are all running, that's probably more than enough, because they only get less than ten percent of the CPU capacity.

But many servers are sleeping most of the time (that is, waiting for requests).

And redundancy is never optimal: 10 copies of an OS is perhaps 100 GB disk space wasted.

3.3. Containers, LXC

A container is like a 'VM', but it borrows as much as possible from the host.

- * OS, applications, management software, libraries, documentation, etc. are used from the host. Only configurations, guest-specific applications and user data. are kept with each CT
- * The running kernel is reused. Linux has mechanism which allow limitation and prioritization of resources (CPU, memory, block I/O, network, etc.)

Specifically: linux supports usage metering, isolating and limiting of key resources such as

- * memory
- * CPU
- * block I/O
- * network

Conceptual building blocks are:

3.3.1. Mount

In linux, a directory can be mounted on another directory. No files are copied for this; the kernel maintains a list of all mount-points and redirects file access on-the-fly. The original directory upon which the new tree is mounted is obscured and is inaccessible but it's still there. As soon as the new directory is unmounted the original is there again.

For instance, directory `/c/d` can be mounted on directory `/a/b` after which a file `/c/d/e` can also be found as `/a/c/d/e`. Directory `/a/b` (and all sub-files and directories) are inaccessible until the directory is unmounted again.

3.3.2. Namespaces

A namespace is an abstraction layer wrapping some global resource. For instance, the list of mount-points mentioned earlier is a namespace available to each process (and not necessarily shared with other processes). In this manner one process can use mounts which another process doesn't have, and by the same token, parts of the file system may be hidden (unmounted) for one process while they are visible to another process.

The following resources are wrapped in a namespace:

- * Mount Mount points
- * Cgroup Metering, Isolation and limitation of resources such as CPU, Memory, Network and Disk I/O
- * IPC Message queues
- * Network Network devices, stacks, ports, etc.
- * PID Process IDs
- * User User and group IDs
- * UTS Hostname and NIS domain name

Namespaces are ideal to support isolation, because a process only has access to specific namespaces and thus to specific parts of the file tree, and specific allocation of resources such as memory, CPU etcetera.

Using namespaces, an LXC only has access to its corner of the world. This limitation is guarded by the kernel and can't (barring security vulnerabilities) be bypassed by the LXC.

Compared to a VM (and apart from user data), an LXC uses less memory (no kernel and fewer buffers) and little disk space

3.4. CT ≠ VM

A VM is data which is being manipulated by a specialized program (the hypervisor) running on top of or embedded in an OS.

A CT is a way of structuring the file system and other OS features such that processes can coexist with their own users, rights, data, etcetera. From their own perspective, CTs are isolated.

Entirely isolated? No: containers are (at this moment) less secure than VMs. See e.g. <http://www.infoworld.com/article/3071679/linux/linux-containers-vs-vm-a-security-comparison.html>.

3.5. LXC Success Factors

Linux containers have almost all advantages of VMs without the major disadvantage: size.

- * Containers can be moved easily => separation of hard- and software, leading to
 - * improved management & operations
 - * scalability
- * improved version control

In conjunction with infrastructure virtualisation we get better security

All in all a powerful current server may sensibly run tens to hundreds of CTs

3.6. Libvirt

Linux containers are surprisingly easy to use. A single library, `libvirt`:

- * supports many VM and LXC solutions: KVM, Qemu, VMWare, Xen, ...
- * supports many hypervisors: LXC, Xen, VirtualBox, VMware, Hyper-V, OpenVZ (LXC precursor)
- * supports many guest OSs: Linux, Windows, Mac OSX, FreeBSD
- * offers APIs in Python, Perl, Ruby, Java, JavaScript(Node), PHP, ...

3.6.1. Type 1 Hypervisors

3.6.1.1. KVM

Kernel-based Virtual Machine (KVM) is a virtualization infrastructure for the Linux kernel that turns it into a hypervisor. A wide variety of guest operating systems work with KVM, including many flavours and versions of Linux, BSD, Solaris, Windows, ReactOS, OS X, and lately Android.

3.6.1.2. VMware

VMware provides cloud computing and platform virtualization software and services. It was the first commercially successful company to virtualize the x86 architecture, and currently offers many desktop and OS virtualization products.

3.6.1.3. Xen

Xen Project is a hypervisor using a microkernel design. Xen Project is currently available for the IA-32, x86-64 and ARM instruction sets.

A microkernel is a tiny OS which offer only the bare minimum of functionality to run as a hypervisor. This functionality includes low-level address space management, thread management, and inter-process communication (IPC).

3.6.2. Type 2 Hypervisors

3.6.2.1. VirtualBox

VirtualBox is a type 2 hypervisor for x86 computers currently being developed by Oracle Corporation. VirtualBox is ported to a number of host operating systems, including: Linux, MacOS, Windows, and others.

3.6.2.2. VMware

VMware Workstation is a type 2 hypervisor that runs on x64 versions of Windows, MacOS and Linux operating systems and allows us to set up virtual machines (VMs) on a single physical machine.

3.6.2.3. Hyper-V

Microsoft Hyper-V is a native type 1 / type 2 hypervisor.

3.7. LXC Alternatives

- * FreeBSD Jails,
- * recently, Windows containers.
- * MacOS / iOS offer sandboxes, similar in effect.
- * **Docker**. Whereas an LXC exposes a full OS with a shell and all installed software, Docker uses LXC to create self-contained environments around single applications. Every Docker CT has precisely what it needs to run one thing, such as a web-server, a CMS, etc.
- * Sandstorm uses LXC to create very fine-grained CTs: every end-user document is opened in its own container.

3.8. Sandstorm

The Sandstorm ecosphere is much smaller than Docker, but it is useful to understand that although very successful, Docker is only one engineering approach

Sandstorm containerizes data objects. Each Sandstorm container manages one single document.

3.9. Docker

Docker has combined LXC and other ideas to such a successful degree, that 'Docker' now is used synonymously with 'virtualization' !

In the next workshop we'll look into Docker.

4. Docker

4.1. Docker

Docker uses LXC and other ideas to offer an enterprise-oriented software governance approach.

4.2. Docker Success Factors

Immutable and **Portable**: once it is created it doesn't change. You can move it or scale it; each instance is identical. It is platform independent; the 'development' version and the 'production' version are identical. Deployment takes seconds. High quality version control.

Once a Docker image has been built, tested and published, it will not be changed. Nor does it have to. If it works in the development environment (using other published components), it will work in the production environment using those same components.

Obviously Docker code is as error-prone as code in other software, so bugs do occur. A fix, however, is published as a new version without impact on existing operational systems. Solutions using this component should start using the new component. Accordingly, enterprise-grade Docker solutions rely on (continuous) integration.

Independent of local files or other applications/containers (except through REST APIs). Dependencies are explicit, leading to lightweight, modular, reusable, scalable, secure solutions.

4.3. Docker Development Process

The Docker Process describes the creation and use, the entire life cycle of Docker-based software.

Inside a Docker container is ordinary software which is designed, developed and (unit) tested in the usual way.

Then, a container is configured and built and integration testing takes place. That is, the container is tested together with other containers in the full stack.

Once (integration) testing is complete, the container is published and can be installed in the operational context.

At this point, one might ask: how about data? We've mentioned Docker containers are portable and immutable, but surely once a container is up and running, data must be collected somewhere.

Simply put, Docker offers several ways to persist data (such as Docker Volumes), which we will discuss shortly.

4.4. Docker Engine

In order to run a docker CT we need a host with the Docker Engine. It can be easily installed on most Linux distros, and some distros exist which are aimed specifically at hosting Docker CTs (e.g. RancherOS). Also, native (Windows and MacOS) implementations are available through docker.com.

Docker Engine fulfills three main functions:

- * Run Docker containers
- * Build Docker containers
- * Manage All running Docker containers on the host, which includes
 - * Networking
 - * Local persistence (Volumes)

4.5. Docker Images

For many purposes, images have already been made. For instance,

```
docker run ubuntu /bin/echo hello world
```

Produces this output:

```
Unable to find image 'ubuntu:latest' locally
latest: Pulling from library/ubuntu
d5c6f90da05d: Pull complete
1300883d87d5: Pull complete
...
dc27a084064f: Pull complete
Digest: sha256:34471448724419596ca4e890496d375801de21b0e67b81a77fd6155ce001edad
Status: Downloaded newer image for ubuntu:latest
hello world
```

4.6. What happened?

- * `docker` starts Docker Engine

- * `run` tries to start a CT. No image for the Ubuntu CT is present so one is downloaded from the online library
- * When that image is started, the line is sent to it
- * `/bin/echo` is a standard Linux command which echoes its input to the output
- * The remainder of the line is offered as input
- * The output is returned to Docker Engine (which echoes it)
- * Having finished the command, the Docker CT is discarded.

Giving the same command again doesn't pull the image again; Docker Engine caches images.

4.7. Example. Grav: simple CMS

Grav is a simple CMS and web server. A Grav Docker CT can be started with:

```
docker run -d -p 80:80 --name grav oleglab/grav:latest
```

The `-d` option runs the CT as a daemon: it will wait for requests on port 80 and will normally never finish.

The `-p 80:80` option tells Docker Engine to connect host port 80 to this container's port 80

But how to add content to this site? One way is to use `docker exec ...` which tells the engine to run a (shell) command in the container.

But less ad-hoc ways exist!

To be precise: `docker exec -it CTname bash` will try to open a bash shell in the container named CTname. 'Try to' because there is no guarantee that CT has bash. Note that most containers have `sh`, so `docker exec -it CTname sh` is a safe bet.

4.8. Docker Hub(s)

Several repositories exist where docker images can be found. Docker Hub is the original repository maintained by Docker inc for the community. github.com/docker-library is also a repository where various 'official images' are maintained (e.g. Python, php, nginx). And individuals and organizations also offer docker images for their products via github.

If an image isn't (yet) available, Docker Engine can make one for you

4.9. Docker Hub (top 9)

 nginx official	6.8K STARS	10M+ PULLS	> DETAILS
 redis official	4.2K STARS	10M+ PULLS	> DETAILS
 busybox official	1.1K STARS	10M+ PULLS	> DETAILS
 alpine official	2.6K STARS	10M+ PULLS	> DETAILS
 ubuntu official	6.6K STARS	10M+ PULLS	> DETAILS
 registry official	1.7K STARS	10M+ PULLS	> DETAILS
 mysql official	5.0K STARS	10M+ PULLS	> DETAILS
 mongo official	3.6K STARS	10M+ PULLS	> DETAILS
 postgres official	4.0K STARS	10M+ PULLS	> DETAILS

4.10. Docker build

```
docker build PATH | URL | -
```

Builds an image from dockerfile and other stuff. URL is github repo (which is recursively fetched), tarball or text file. `-` means read from stdin.

4.11. Dockerfile

A **dockerfile** describes how a CT image is built. Here, mongodb (noSQL) server is built from a clean Ubuntu server. Use the `docker build ...` to build an image using this docker file.

```
FROM ubuntu
MAINTAINER Kimbro Staken
RUN apt-key adv --keyserver keyserver.ubuntu.com --recv 7F0CEB10
RUN echo "deb http://downloads-distro.mongodb.org/repo/ubuntu-upstart dist 10gen" |
tee -a /etc/apt/sources.list.d/10gen.list
RUN apt-get update
RUN apt-get -y install apt-utils
RUN apt-get -y install mongodb-10gen
CMD ["/usr/bin/mongod"]
```

Note: building images can be confusing because we have to think simultaneously at three levels (linux machines):

- * the machine that executes the build process
- * the container as it is being built
- * the future container when it is running

The build process follows a straightforward pattern: it maintains a container in which it executes one command from the Dockerfile. That results in the next container, in which the next line is applied, and so forth.

A brief overview of Dockerfile commands (note that this doesn't substitute the actual manual; more commands and varieties of commands exist).

4.11.1.1. FROM name

start building with a fresh image (name identifies it in the repository)

4.11.1.2. RUN command

execute this command in the CT being built

4.11.1.3. Label key=value ...

Add meta-data to the container

4.11.1.4. EXPOSE port ...

Exposes ports on the container. Obviously, processes within the container should handle communication on these ports. All other ports are closed and inaccessible.

4.11.1.5. ENV key=value ...

Set environment variables in the built container.

4.11.1.6. ADD src ... dest

Copy files from the machine used to build the image to the container that is being built. This is the way to get static content into an image. Note that `src` may be an archive (which is then unpacked), or an URL (the content of which is fetched).

A command `COPY` exists with similar meaning which is less versatile and therefore less used.

4.11.1.7. CMD command

when the image is finished and an instance is started, this is the command that that it will execute.

Note that this is at most one command, often to start the may service. If more than one commands are needed they can be put in an executable shell script.

4.11.1.8. ENTRYPOINT

when the image is finished and an instance is started, this is the command that that it will execute.

Note that this is at most one command, often to start the may service. If more than one commands are needed they can be put in an executable shell script.

Confused? Rightly so!

When you execute `docker run image` without arguments, either `CMD` or `ENTRYPOINT` must be defined, and will be used.

A CMD command can be simply overridden by the user starting the image. ENTRYPOINT can also be overridden, but slightly less easy. So CMD is used for a default intended to be overridden, whereas ENTRYPOINT signifies 'normal' operation but allows overriding.

CMD and ENTRYPOINT can both be used, allowing CMD to offer default arguments for ENTRYPOINT.

4.11.1.9. VOLUME directory

Creates a mount point in the container being built. When the image is instantiated, Docker Engine creates a persistent volume and mounts in here. A volume survives the end (and/or restart) of this container. See below for more information on volumes.

4.11.1.10. USER

Sets the user and group to use when running the image

4.11.1.11. WORKDIR

Sets the working directory in which subsequent RUN commands are executed

4.12. Docker-compose

The `docker run ...` command works well for individual containers, but some applications consist of several cooperating containers.

When `docker run` gets overly complex, `docker-compose` can be used. It uses a structured configuration-file (using YAML) to describe which containers, volumes and networks should be created, and how.

4.13. Getting Started 1

First: get a docker engine (as a production system) running in an environment that you are comfortable with.

- * Since a year, docker engine exists natively on Windows and Mac.
- * Alternatively, you can create your favorite Linux distro under virtualbox (or another VM platform) and install docker there.
- * Or you can accept a challenge and install RancherOS or CoreOS on virtualbox and use that as a production system.

4.14. Getting Started 2

Separately from this you need a development environment. Interestingly, Docker CTs are available for this. So: use the environment that you are comfortable with; get a development container; or create an environment to do this.

Read about [Consistent Cross-Platform Docker Development Environments](#)

4.15. Getting Started 3

Follow [these instructions](#) to create your first docker CT

4.16. Isolation: UnionFS

UnionFS is a Linux/Unix file system which allows files and directories of separate file systems (branches) to be transparently overlaid. Different branches may be RO or RW.

For instance, a full bootable OS on a stick contains a RO branch on the stick and a RW branch in memory, so that users can try out everything (but loose changes when the computer is turned off).

Every Docker container starts with the RO branch in the image overlaid with a RW branch for all running data. That branch is lost with the end of the CT.

For instance, suppose files `/a/b` and `/a/c` exist and suppose that the first directory `a` is RO and is overlaid RW with the second directory `a`. In the unified directory `b` and `c` are present. If `b` is edited a copy of `b` is made and changed; the original remains unchanged (and is inaccessible from the unified file system). If the file `c` is changed, those changes are also visible in the original directory `/a/`. When a new file `d` is created in `a`, that file appears in the unified directory but also in the directory in which `c` appeared.

4.17. Persistence

A CT may have private data, but when it crashes all is lost. Docker CTs offer no persistence...

https://www.youtube.com/watch?v=4o0WYS32x_E

A Docker **volume** is a file or directory that exists independent of other containers, that can be mounted in any container (when it is started) and that isn't deleted or changed when the CT is removed or reinstalled.

```
docker run -v [src:]dst[opt] ...
```

declares `dst` to be volume in the container (i.e. persistent). If `src` is a directory, that directory in the host is used; otherwise a directory is created. If `src` is a name, another container may share the same volume (effectively using a shared directory). Options can make a volume to be mounted RO.

Note that volumes allow data to survive the container, but:

- * when the container is moved to another host in a cluster, the data isn't moved automatically. Volumes require additional care to allow for portability.
- * when the host crashes data can still be lost. However, backup solutions exist which backup volumes and host directories to external stores.

4.18. Networking

watch <https://www.youtube.com/watch?v=RCG-5N41FpQ>

Docker offers four networking modes:

- * `host` : the CT uses the host NIC directly
- * `bridge` : the CT uses an internal network which can be bridged (using NAT) to the host. (This is the default)
- * `none` : the CT uses its own network without a bridge

* `overlay` : for multi-host solutions; hosts communicate with a key-value store to configure an overlay network (based on VXLAN)

5. Intermezzo: REST

5.1. Internet, WWW, HTTP, REST

Around 1960 research began on packet switching, and around 1970 '*the Internet*' originated from a growing number of interconnected local area networks. In 1982 the first versions of TCP/IP were developed, and networks around the world could easily join this 'Internet'.

In 1989 the World Wide Web was conceived as a (global) distributed information system with readable text documents containing hyperlinks: references to directly relevant other documents. At the time, internet was primarily used by researchers, and the idea of all knowledge being immediately available and interrelated without libraries, printers and mail was breathtaking!

From 1990 HTTP was developed, to be formalised in v1.0 in 1996.

Around 2000, one of the developers, Roy Fielding, worked on v1.1 and created the architectural model REST (Representational State Transfer). V1.0 reigned until 2015, and even now, HTTP/2 is a pure extension. REST still rules.

The name 'Representational State Transfer' suggests an application being seen by the user as a collection of web pages, where some action on one page (such as clicking a link) transfers the user to another page, which at that moment represents the new state of the application: Representational State Transfer.

Note that REST is at the basis of HTTP, but a server using HTTP doesn't necessarily implement REST. If it does, it is called **RESTful**. The reverse is also true: a RESTful system could be made using something else than HTTP. In practice, the terms are often used interchangeably.

5.2. Presentation, Tiers and MV*

So far, the word 'presentation' hasn't been used. HTTP doesn't say what to do with the data. But the best known example is web-browsing, where the data concerns HTML and multi-media. For instance, when you enter an `http://` -URL in your browser, the browser will send an HTTP GET message, and when you upload an HTML form, the browser will send an HTTP POST message.

But HTTP is also used by back-end systems exchanging information.

5.2.1. MV*

With the exception of simple websites, many modern websites are structured using the MV* pattern (which includes MVC, MVP and others). Here, M stands for Model and V for View (and C for control, P for presenter, etcetera).

'View' refers to the part responsible for presenting the information. It knows about themes, colors, placement, etcetera.

'Model' refers to the part responsible for storing and manipulating information. It knows about users, credentials, lists of data, etcetera.

(in the MVC/MVP patterns 'Control', refers to the part which reacts to GUI action by changing the model, and the 'Presenter' is also responsible for generating new static views when the model changes)

In very large installations, the different concerns may be implemented on different servers. The 'view' server is a webserver which talks HTTP to the end-user. But the 'model' server might be a data-server which doesn't talk to the end-user at all.

How then do the view-server and the model server talk to each other? HTTP is well suited, and many of the 'restful' characteristics described below are useful.

5.2.2. Tiers

Large applications are often designed using the multi-tier design patterns (which predates internet). A 3-tier system might have tiers (or layers) for Presentation, Data and Business Logic.

Presentation (like View) is responsible for what the end-user (employee, customer, ...) sees. Data (similar to Model) is responsible for storing information, for instance in databases. The middle tier is called 'Business Logic' because in it the rules and procedures of the organisation are codified.

These three tiers seem like MV*, but MV* addresses presentation, and 'controls' or 'presenters' are very small, concerned only with updating the view-model based on UI action and *not* with business logic at large. Very often the presentation tier is itself structured using the MV* pattern.

The wisdom in these tiers is obvious:

- * marketers like to change the looks of an application with some regularity. Separating this concern in one tier means that the other tiers are untouched by frequent visual improvements.
- * databases grow and need continuous updates and maintenance. Separating this concern in the data layer ensures that higher tiers are stable across significant changes below.
- * business logic changes only if there are procedural changes (different strategies, organisational change, etcetera).

In large organisations, the different layers are often implemented on different systems, and HTTP is a well suited communication medium between the layers.

5.3. HTTP

HTTP concerns resource management (making, fetching, changing, destroying). In the spirit of REST, the name of the Hypertext Transfer Protocol (HTTP) isn't so much about transferring hypertext as it is about transferring the user to another application state.

From this, we can deduce the general HTTP pattern. The user initiates an action, which results in a HTTP message being sent to the server. The server does its thing and responds with the new application state. Restful systems are stateless, so the server doesn't 'remember' this application state. (If anything is to be remembered, it must be stored in the client; hence, cookies)

5.3.1. URL, URI, URN

HTTP doesn't describe *what kind of* resources are managed, but it is concerned with *where they are located*. A URL (Uniform Resource Locator) is an important notion for the web. A URI (... Identifier) is a string which identifies a name or a resource on the internet. A URL is a special URI, which not only defines the name, but also the way to access the resource. For instance: `http://rooster.hva.nl` is a

URL, whereas `rooster.hva.nl` is not. A URI which is not a URL but just a name is called a URN (... Name).

5.3.2. Verbs

The action effect is expressed in an HTTP message's **verb**. Verbs include:

- * **GET**: fetch the resource identified by the URI. The response will include the resource (such as the contents of an HTML file), or, if it is an action, the result produced by that action (such as the output of an executed PHP file).
- * **PUT**: change an existing resource or create a new one (if the client may choose the new URI (if the server should choose new URI's, use POST)).
- * **POST**: create a new resource. The response will include the URI for the new resource.
- * **DELETE**: destroy the resource.

5.3.3. Restfull Applications

Making applications restfull has many advantages. For instance, one of the requirements is that a GET message doesn't have side effects. If that is the case, it is possible to achieve scaling by allowing many servers to handle GET messages. Side-effect would complicate that approach significantly.

REST constraints include:

- * Uniform Interface
HTTP offers an interface for creating, fetching, changing and destroying arbitrary resources across a TCP/IP network. Using the uniform interface allows for the independent development and maintenance of components and subsystems.
- * Stateless
REST servers are stateless: there is no need to think about sessions or other transactions. Scaling can be achieved by parallelism (from threads to servers).
- * Cacheable
REST should be cacheable. Since GET doesn't have side effects, the result can be cached until the resource itself changes, greatly reducing net traffic. This is a key advantage of REST over RPC (Remote Procedure Calls).

5.4. Further Reading

More on HTTP and multi-tier systems in the 'WorkshopREST'

An interesting guide can be found in http://www.restapitutorial.com/media/RESTful_Best_Practices-v1_1.pdf

6. Practicum

Door omstandigheden gaan we verder in het Nederlands

Titel: Rest Servers Practicum: Tit for Tat

6.1. Rest Servers: Prisoner's Dilemma & Tit-for-Tat

In dit practicum zullen we een aantal eenvoudige REST-servers ontwikkelen en testen en we zullen ze bouwen en in gebruik nemen als docker containers.

6.2. Inleiding

Het Prisoner's Dilemma is een bekend filosofisch vraagstuk. De naam komt van twee verdachten die onafhankelijk ondervraagd worden. Als beide niets zeggen kunnen ze hooguit voor medeplichtigheid bestraft worden, zeg 5 jaar elk. Maar als de ene de andere aanwijst als dader krijgt die (zeg) 15 jaar en de ander krijgt geen straf.

Het Iteratieve Prisoner's Dilemma is een idee uit de speltheorie. Twee handelaren wisselen herhaald goederen uit. Dat doen ze door de goederen naar de ander te sturen. Beide kunnen de goederen die ze ontvangen pas bekijken nadat ze hun eigen inbreng verstuurd hebben. Verder communiceren ze niet.

Als ze allebij goederen sturen is dat voor beide gunstig want ze kunnen de goederen lokaal met winst verkopen (zeg +10). Maar als de ene wel iets stuurt en de ander niet is dat voor die ene zeer ongunstig en voor de ander zeer gunstig (zeg, +50 resp. -50). Als ze beide niets sturen gaan ze er niet op voor- of achteruit (± 0).

Rond 1980 is er een groot experiment uitgevoerd rond het iteratieve prisoner's dilemma. Deelnemers konden een programma inzenden waarin de strategie van een Handelaar staat. Vervolgens werden er heel veel rondes gespeeld waarin steeds twee handelaren handelen zoals boven beschreven. De vraag was: welke strategie levert op den duur de meeste winst. De winnaar (uit heel veel ingezonden strategieën) was *Tit-for-Tat*.

- * *Tit-for-tat* is constructief en zal in eerste instantie goederen sturen
- * Maar als een opponent geen goederen stuurt zal *tit-for-tat* dat in de volgende ronde ook niet doen
- * Zolang de opponent geen goederen stuurt zal *tit-for-tat* dat ook niet doen.
- * Zodra de opponent goederen stuurt zal *tit-for-tat* dat (in de volgende ronde) ook weer doen
- * Dit komt er op neer dat *tit-for-tat* het gedrag van de opponent uit de vorige ronde kopieert

In dit practicum gaan we dit experiment bouwen. Dat wil zeggen, we bouwen een aantal servers die zich als handelaren gedragen en we meten gedurende enige tijd hoe succesvol elke Handelaar is ten opzichte van de andere handelaren.

6.3. Servers die het experiment uitvoeren

We kunnen het experiment bouwen met drie soorten servers:

- * de Game Master (die handelaren in contact brengt en die statistieken bijhoudt)
- * de Handelaren
- * Postbussen (die steeds de handel tussen twee handelaren reguleren)

6.3.1. Handelaar

Een Handelaar vertoont het volgende gedrag:

- * meld je aan bij de Game Master
- * (*) een Postbus brengt je in contact met een andere handelaar
- * start een handels-sessie. In elke sessie

- * besluit of je wel of niet handelswaar stuurt aan de Postbus
- * van de postbus krijg je te horen of de andere handelaar handelswaar heeft gestuurd
- * van de postbus krijgt de handelaar te horen of de sessie doorgaat of wordt beëindigd
- * als ie doorgaat: herhaal de stappen in de sessie
- * de sessie is beëindigd; wacht op een bericht van een postbus (*)

6.3.2. Postbus

Een postbus vertoont het volgende cyclische gedrag:

- * meldt zich aan bij de Game Master
- * krijgt stuur-informatie zoals adressen van twee handelaren en de lengte van een sessie
- * start een handelssessie
 - * krijgt van twee handelaren wel of niet handelswaar
 - * geeft het resultaat door aan de handelaren
 - * stelt de sessie-statistiek bij
 - * als de sessie doorgaat: herhaal de stappen
- * De sessie is beëindigd; stuur statistieken aan de Game Master
- * herhaal

6.3.3. Game Master

De Game Master vertoont het volgende cyclische gedrag:

- * accepteer verzoeken van Postbussen
 - * als de postbus statistieken rapporteert, verwerkt deze in de globale telling
 - * plaats inactieve postbussen in een queue
- * accepteer verzoeken van Handelaren en plaatst ze in een queue
- * Als de queue met handelaren minstens 2 lang is en de queue met postbussen minstens 1:
 - * maak ze in contact en start de sessie
- * accepteer een verzoek van ons om de statistieken tot nu toe te geven
- * accepteer een verzoek van ons om de hele sessie te herstarten

6.4. Functioneel en Technisch Ontwerp GameMaster

Als eerste component zullen we de Game Master bouwen, omdat de andere componenten pas getest kunnen worden als de Game Master reeds bestaat.

6.4.1. Game Master Services

Uit de beschrijving van de Game Master halen we de volgende services:

- * `/handelaar`
Een Handelaar meldt zich aan
- * `/postbus`
Een Postbus meldt zich voor dienst
- * `/statistics`
Een postbus geeft statistieken van de laatste sessie

- * `/report`

Rapporteer alle statistieken tot nu toe

- * `/reset`

Zet alle statistieken op nul en stuur alle Handelaren en Postbussen een `reset` opdracht

Het moet je opvallen dat van deze services, de `report` service zinvol gebruikt kan worden nog voordat de andere servers draaien. We kiezen dus die service als eerste om de server te testen.

6.4.2. Game Master Data

Uit de beschrijving van de Game Master halen we dat deze de volgende data moet bijhouden:

- * Handelaren

Een lijst van alle handelaren (moeten we bijhouden om de `reset` boodschap te kunnen sturen)

- * Postbussen

Een lijst van alle postbussen (moeten we bijhouden om de `reset` boodschap te kunnen sturen)

- * InactieveHandelaren

Een lijst van alle inactieve handelaren (moeten we bijhouden om een sessie te kunnen starten)

- * InactievePostbussen

Een lijst van alle inactieve postbussen (moeten we bijhouden om een sessie te kunnen starten)

- * Statistieken

- * Aantal ronden totaal

- * Per Handelaar:

- * Aantal ronden

- * Winst/verlies

- * Constanten

In het experiment worden enkele waarden gebruikt door alle componenten: het aantal ronden per handels-sessie, winst/verlies waarden. Een goede oplossing is, om deze waarden via de Game Master aan de componenten te geven.

6.5. Intermezzo -- Ontwikkelomgeving

Een Docker container is een applicatie die als container is verpakt.

Voordat je de container bouwt moet je eerst zorgen dat de applicatie werkt. Je moet eerst een ontwikkelomgeving voor de applicatie (een REST server) hebben.

- * Als je in JavaScript ontwikkelt:

- * download nodejs en nvm van [<https://nodejs.org/en/download/>]
(<https://nodejs.org/en/download/>)

Nodejs biedt de basis om HTTP servers te bouwen, maar we gebruiken ook nog een library om het voetwerk te verrichten. Er zijn meerdere libraries om te kiezen: express, restify, koa, hapi

- * Express is oud en vertrouwd

- * Restify is de snelste maar leidde ooit vaak onder versie-perikelen

- * Koa is mooi maar gebruikte ooit nieuwe JavaScript technieken die nog niet overal ingezet werden

- * Hapi is de langzaamste maar leidt tot inzichtelijke code.

- * Kies een platform waarbij je je thuis voelt
- * Als je in Python ontwikkelt:
 - * Download Python <https://www.python.org/downloads/>
 Let op: veel systemen zoals macOS hebben al een Python versie die in het OS zelf gebruikt wordt. Dat is vaak een oude versie van Python. Laat deze versie staan en installeer daarnaast de nieuwe Python 3.6 (?)
 Ook bij Python gebruiken we een raamwerk als basis
 - * Bottle is een minimale oplossing geschikt voor kleine servers
 - * Flask is meer uitgebreid en uitbreidbaar en is geschikt voor veel situaties
 - * Django is een groot raamwerk dat in enterprise oplossingen gebruikt wordt

De eerste server zullen we als voorbeeld bouwen in JavaScript met Hapi. De volgende server doen we met Python en Bottle. Maar doe jij in wat voor jou werkt!

6.6. Adressering

Bij het testen en uitrollen via Docker van REST Api's moeten we even over adressering (IP nummers en poorten) nadenken.

- * Op het ontwikkelplatform hebben alle servers het zelfde IP nummer (namelijk dat van de server), en moeten ze dus verschillende poorten gebruiken om te communiceren
- * In de Docker omgeving hebben alle servers verschillende IP nummers om te communiceren, al dan niet met verschillende poorten

De IP nummers van de Docker containers zijn van te voren niet bekend.

In onze opzet moet één IP nummer bekend zijn: dat van de GameMaster. Immers, de andere servers melden zich eerst bij de GameMaster en geven daarbij hun actuele IP nummer door.

Hoe het adres van de GameMaster wordt doorgegeven aan de andere servers is via een 'Environment Variable'. In Linux en macOS zijn alle zogenaamde Shell Variables toegankelijk als environment variable. Een shell variable krijgt als volgt een waarde die in een server 'gelezen' kan worden.

```
export naam=waarde
```

We gebruiken dit mechanisme op twee manieren.

- * het (GameMaster) poort nummer. Het is niet handig om dit in de source-code van de GameMaster te plaatsen, want de andere servers hebben het ook nodig. Om die reden bepalen we het van te voren door in de shell in te geven: `export GameMasterPort=10000`. Nu kennen alle servers het poort-nummer van de GameMaster.
- * het (GameMaster) IP nummer. Het is lastig dit nummer van te voren te bepalen, zeker aangezien het tijdens de test-fase een ander nummer is dan tijdens de 'Docker' fase. Om die reden rapporteert de GameMaster z'n IP nummer zodra hij 'draait' door 'm af te drukken. (het is niet echt mogelijk hier shell-variabelen voor te gebruiken die alleen doorgegeven worden aan sub-shells; maar output van een proces wordt wel doorgegeven aan de parent). Door deze output op te vangen en ook in een shell variabele te stoppen wordt ook het IP nummer doorgegeven.

6.7. Game Master V1

Dit is versie 1 van de GameMaster server:

```
<<(GameMasterV1.js)
```

6.7.1. Eerste Test

Testen doe je door de server te starten en met een POST-tool een bericht te posten. De eerste keer dat je de server start krijg je zoiets als

```
$ export GameMasterPort=10000
$ node GameMaster.js
module.js:491
    throw err;
    ^

Error: Cannot find module 'hapi'
    at Function.Module._resolveFilename (module.js:489:15)
    ...
```

In de code staat `require('hapi')` maar we hebben die module nog niet op ons systeem geïnstalleerd. Dat doe je met

```
npm install hapi
```

Doe dat voor elke module die je `require-t`.

Als alles goed gaat zegt de server

```
<ip-van-je-laptop>
```

Om de server te testen hebben we een POST tool nodig, zoals <https://www.getpostman.com/> (alle platformen)

Met het POST tool sturen we nu een bericht aan `http://<ip-van-je-laptop>:10000/report`. We laten de body leeg (het maakt voor deze service niet uit wat daarin zit).

Als alles goed gaat is het antwoord:

```
{
  "sessions": 0,
  "profits": {}
}
```

6.8. FO en TO Handelaar + Postbus

Als 2e component maken we de Handelaar.

6.8.1. Handelaar Services

* => /handelaar

Geen service die de Handelaar zelf biedt, maar een service die de handelaar aanroept op de GameMaster.

* /startSession

Een postbus geeft de handelaar het startsein van een sessie. De handelaar en de postbus zullen nu de volgende berichten uitwisselen:

* => /goods { "goods" : true/false }

De handelaar stuurt een bericht aan de postbus en geeft aan of de handelswaar wel of niet gestuurd wordt.

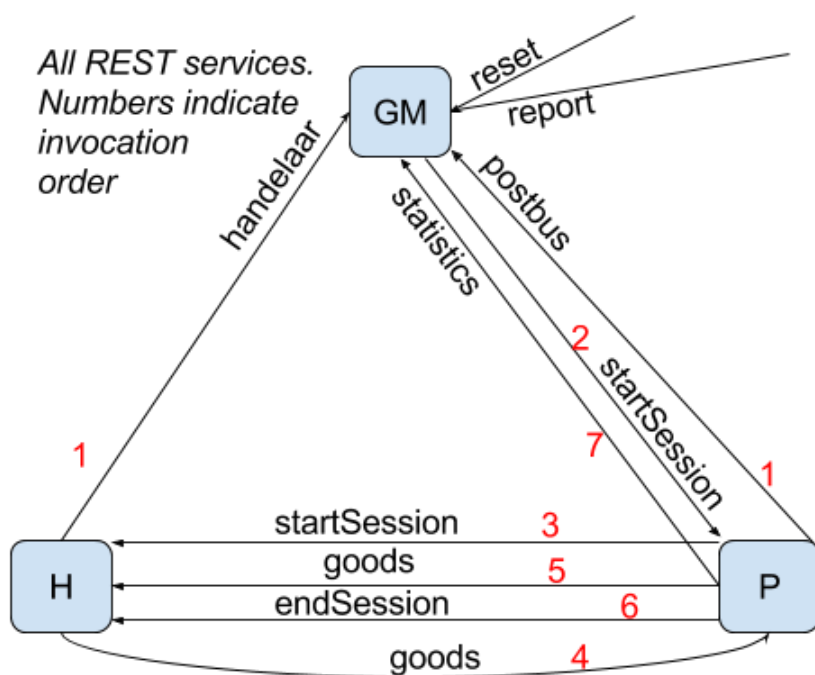
* /goods { "goods" : true/false }

De postbus zegt tegen de handelaar of de andere handelaar wel of niet handelswaar heeft gestuurd

* /endSession

De postbus zegt de handelaar dat de sessie is afgesloten

6.8.2. Postbus Services



* => /postbus

Geen service die de Postbus zelf biedt, maar een service die de postbus aanroept op de GameMaster.

* /startSession { "one": {

De GameMaster geeft een postbus het startsein van een sessie (en geeft twee handelaren door aan de postbus). De handelaren en de postbus zullen nu de volgende berichten uitwisselen:

* /goods { "goods" : true/false }

De handelaar stuurt een bericht aan de postbus en geeft aan of de handelswaar wel of niet gestuurd wordt.

* => /goods { "goods" : true/false }

De postbus zegt tegen de handelaar of de andere handelaar wel of niet handelswaar heeft

gestuurd

* /endSession

De postbus zegt de handelaar dat de sessie is afgesloten

* => /statistics

Geen service die de Postbus zelf biedt, maar een service die de postbus aanroept op de GameMaster.

6.9. HandelaarV1, PostbusV1 en GameMasterV2

Voordat we de Handelaar en Postbus kunnen testen moet de GameMaster iets verder uitgebreid zijn. Deze versie implementeert twee extra services: `/handelaar` en `/postbus`:

<<(GameMasterV2.js)

Deze Handelaar gaan we in Python maken, op basis van bottle. Bottle installeer je met `pip install bottle`.

Om tussentijds testen mogelijk te maken, implementeren we nog geen services. Deze Handelaar meldt zich aan, maar accepteert maar één service-call: `startSession`. Als dit bericht wordt ontvangen (overeenkomstig met nummer 2 in de figuur hierboven) wordt alleen een boodschap afgedrukt.

De Handelaar moet wel het IP nummer en poort van de GameMaster server hebben. Om die reden definiëren we een shell-variabele om alle servers op de hoogte te stellen van de poort van de GameMaster. We voeren in de shell in `export GameMasterPort=10000`.

Het IP adres van de GameMaster is pas bekend als die server wordt gestart. Om die reden print de server het eigen IP adres nadat deze gestart is. Door dat adres op te vangen kunnen we het aan andere servers doorgeven. We starten de GameMaster straks op met:

```
export GameMasterPort=10000; export GameMasterIP=$(node GameMasterV2.js)
```

<<(HandelaarV1.py)

Ook de eerste test met Python programma's kan een foutmelding geven over niet-bestaande modules.

```
$ python3 HandelaarV1.py
Traceback (most recent call last):
  File "HandelaarV1.py", line 1, in <module>
    import requests
ModuleNotFoundError: No module named 'requests'
```

Python modules worden ingeladen met e.g. `pip install requests`

We kunnen onderzoeken of de Handelaar zich goed heeft aangemeld door het bericht `http://192.168.0.125:10000/report` te POSTen.

De derde server is een Postbus. De code lijkt erg op die van de Handelaar (vooralsnog, omdat er nog geen services zijn geïmplementeerd).

<<(PostbusV1.py)

6.10. Intermezzo -- Shell Script

Het kleinschalig ontwikkelen en testen van REST servers is wat lastiger dan van 'normale' software omdat REST servers inherent parallel werken en in principe geen (eigen) UI hebben. Om toch makkelijk te kunnen testen, kunnen we een shell-script maken dat een paar pijnpunten opvangt.

In dat shell script moeten we alle servers starten en moeten we zorgen dat eventuele output (JavaScript `console.log` of Python `print` opdrachten die we toevoegen tijdens het testen) netjes en overzichtelijk getoond wordt.

Maak een file met de naam `tit4tat.sh` met de volgende inhoud:

```
<<(tit4tatV1.sh)
```

Je kan de file executeren met `sh tit4tat.sh` maar dat is voor watjes. Linux kenners maken de file executable door in de shell `chmod u+x tit4tat.sh` in te tikken. Daarna kan het script uitgevoerd worden met `tit4tat.sh`.

Toelichting

1. Zoals besproken gebruiken we een shell variabele om de poort van de GameMaster aan alle servers te geven.
2. Een moeilijkheid bij het ontwikkelen van REST servers is dat ze in de achtergrond draaien en daar hun poort vasthouden. Als je moet herstarten tijdens het ontwikkelen, moet je soms die processen opzoeken en `kill` en. In dit script bewaren we alle process id's in een file, en we definiëren een `trap` waarin al die processen gekill-ed worden. De trap word uitgevoerd als we dit script, bijvoorbeeld met `^C`, killen.
3. Hier wordt de GameMaster gestart en het process id in file `pids` gestopt (`$!` is altijd de pid van het laatst gestarte process).
In Linux-achtige omgevingen hebben processen standaard twee output-files: (1) voor standaard output, en (2) voor foutmeldingen. Wij willen alle output bij elkaar hebben (want we willen niet vele logs bekijken). De uitdrukking `2>&1` zegt zoveel als: dupliceer alle foutmeldingen naar de normale output. Vervolgens zegt `> out`: schrijf alle output (en dus ook foutmeldingen) naar een file `out`. Het `&` teken aan het eind van de regel zegt: wacht niet tot het commando klaar is (want het is een server die altijd moet draaien) maar voer het commando in de achtergrond uit.
4. We hebben het IP nummer van de server nodig maar kunnen dat niet meteen doen, want de server is nog bezig met opstarten. Het commando `tail -f out` pakt regels uit die file, zonder ooit te stoppen (`-f` van *follow*). Dus dat stoppen moeten wij doen. Dat doen we met `kill $!`. Die opdracht wordt pas uitgevoerd als we het IP nummer uit de file gehaald hebben. Dat doen we met `grep -o` (geef alleen wat je vind als output) `-m 1` (stop zodra je een match hebt) `'[0-9.]\{7,15\}'` (zoek iets dat lijkt op een IP nummer).
Het resultaat schrijven we in een file `ip` met `>ip`.
De constructie `<( ... )` voert een commando uit in een subshell en geeft de output door als input aan het commando links.
In de laatste regel van dit stanza wordt de inhoud van de file `ip` in de variabele `GameMasterIP` gestopt.
5. De volgende drie stanza's zijn nu recht door zee: start een server en voeg alle output toe achteraan file `out` (`>>`)
6. De laatste stanza zorg er voor dat alle output naar de console word gekopieerd.
7. Je zou nu de gedachte kunnen krijgen: 'waaaaauw wat moeilijk, dat kan ik nooit'. Maar ik ken niemand die dit script in één keer opschrijft, en een aantal dingen heb ik voor het schrijven van

deze handout ook nog nooit gebruikt. Een script maken kost tijd en brengt je veelvuldig langs Google, StackExchange c.s. en meer exotische websites. Maar: uiteindelijk scheelt een script handwerk en win je de tijd terug.

6.11. GameMasterV2a

Het automatiseren van het starten van de servers brengt meteen een bug aan het licht in GameMasterV2: Handelaar en Postbus melden zich aan met de `/handelaar` en `/postbus` boodschap en krijgen dan van de GameMaster een poort nummer. Pas daarna, bijvoorbeeld 0.1 seconde later, starten ze de REST service op die poort. Maar ondertussen is de GameMaster al verder gesneld. Als er twee handelaren en één postbus zijn, wordt de functie `startSession` uitgevoerd en wordt er een boodschap gestuurd nog voordat de servers klaar zijn. Die boodschap wordt nooit ontvangen.

Een goede oplossing hiervoor is om een extra service toe te voegen: één voor de initialisatie, en één voor het starten zodra de server er klaar voor is.

Het is niet altijd mogelijk om services toe te voegen. Een andere methode die in de praktijk veel wordt toegepast is een korte wachttijd in te voegen zodat de server gereed is.

In GameMaster2a zijn de regels `startSession()` vervangen door `setTimeout(startSession, x);`. In JavaScript roept `setTimeout` asynchroon een functie aan nadat een timer is afgelopen. Voor `x` wordt een waarde (in miliseconden) gebruikt die lijkt te werken.

We hebben de boodschap `startSession` nummer 2 uit de figuur ontvangen en willen nu `startSession` nummer 3 zenden en ontvangen.

6.12. Opmerkingen

De gebruikelijke softwareontwikkelmethoden zijn natuurlijk van toepassing op REST servers. In kleine projecten (zoals dit project) zijn de volgende richtlijnen nuttig:

- * Kleine stappen

Test steeds na een kleine toevoeging. Zo klein mogelijk. Als je test na meerdere aanpassingen en je systeem doet het niet, waar ligt dat dan aan? Hoe meer stappen tegelijk, hoe groter de onzekerheid. Dus.

- * Bekijk de gegevens

We hebben de uitvoer van alle servers naar de console gestuurd zodat je makkelijk output kan bekijken. Als je een stap toevoegt en er gebeurt niet wat je had verwacht, is het zaak `print` of `console.log` statements rond je aanpassing neer te zetten. Is alles zoals je denkt dat het is. Markeer in die statements duidelijk waar ze staan, en haal de statements weg (of maak ze commentaar) als het probleem is opgelost.

- * Zet in Python achter elke print die je gebruikt tijdens het debuggen: `sys.stdout.flush()`. Output wordt in Python namelijk gebufferd, maar voor tracing doeleinden is dat alleen maar lastig. Flush leegt de buffers en print de inhoud meteen.

- * Tijdens het testen merk ik regelmatig dat REST messages soms zoekraken. Mijn indruk is: vooral als ik het script stop en meteen weer start. Poorten worden beheerd door het OS. Als een proces gestopt is, moet het OS de in gebruik zijnde poorten nog opruimen. Als je een nieuw proces start met dezelfde poort, kan het zijn dat boodschappen zoek raken. In principe ontstaat er ergens een foutmelding, maar of Python en/of Node die altijd goed doorgeven is de vraag.

6.13. HandelaarV2

Eindelijk is het zo ver en ontwikkelen we de kern van de handelaar.

In de inleiding zagen we dat de handelaar *tit-for-tat* in eerste instantie goederen zal sturen en vervolgens steeds het gedrag van de handelspartner zal kopiëren. Dat betekent dat de handelaar één Boolean `willTrade` bijhoudt die de status van de vorige ronde weergeeft.

- * In `startSession` krijgt `willTrade` de waarde `True`
- * In `goods` (4) wordt deze waarde gestuurd
- * In `goods` (5) wordt de waarde van de andere handelaar in `willTrade` geplaatst

De tussentijdse en de laatste message in een handelssessie zijn precies hetzelfde omdat deze handelaar verder geen persistente data hoeft op te slaat.

<<(HandelaarV2.py)

6.14. PostbusV3

PostbusV3 moet iets meer bijhouden: per handelaar de inzet van die ronde.

<<(PostbusV2.py)

6.15. Intermezzo: WSGI server

Tijdens het testen merk ik een volgend probleem: de standaard WSGI server (Web Server Gateway Interface, de standaard voor Python web application development) is alleen voor ontwikkeling bedoeld.

Wat ik merk is dat als handelaar en postbus al te voortvarend boodschappen sturen, die niet meer beantwoord worden.

Een betere server is `paste`. Dus: `pip install paste` en dan de sever starten met `run(host=me['myIP'], port=me['port'], server='paste', debug=True)`.

Deze handout is nog niet af maar nadert voltooiing.